COMP9414 Artificial Intelligence

Tutorial topic: Graph Search with ai9414

UNSW Sydney

1 Introduction

We explore four graph search algorithms through the browser-based `ai9414` teaching demos:

- `graph-dfs`: depth-first search on an unweighted spatial graph.
- `graph-bfs`: breadth-first search on an unweighted spatial graph.
- `graph-ucs`: uniform-cost search on a weighted spatial graph.
- `graph-astar`: A* search on a weighted spatial graph.

The aim is to connect the algorithmic idea to the state of the frontier shown in the visualisation. DFS and BFS differ only in the data structure used to organise the frontier. UCS and A* use a priority queue, but they assign different priorities to frontier paths.

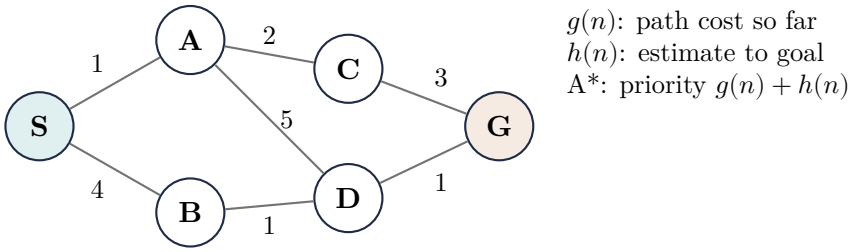

Figure 1: A small weighted graph. DFS/BFS reason about reachability and depth; UCS/A* reason about path cost.

2 Installation and Setup

2.1 Create a Python environment

The package requires Python 3.11 or newer. On macOS or Linux, create and activate a virtual environment as follows:

```
mkdir ai9414-search-tutorial
cd ai9414-search-tutorial
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install ai9414
```

On Windows PowerShell, the activation command is usually:

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install ai9414
```

Check that the command-line tool is available:

```
ai9414 list
```

If the command is not on your `PATH`, use the module entry point:

```
python -m ai9414 list
```

2.2 Run the graph demos

Each command starts a local web app and opens a browser window. Leave the command running while you use the demo, then stop it with `Ctrl-C`.

```
ai9414 demo graph-dfs --example small
ai9414 demo graph-bfs --example small
ai9414 demo graph-ucs --example small
ai9414 demo graph-astar --example small
```

To see the available examples for one demo:

```
ai9414 list --examples graph-astar
```

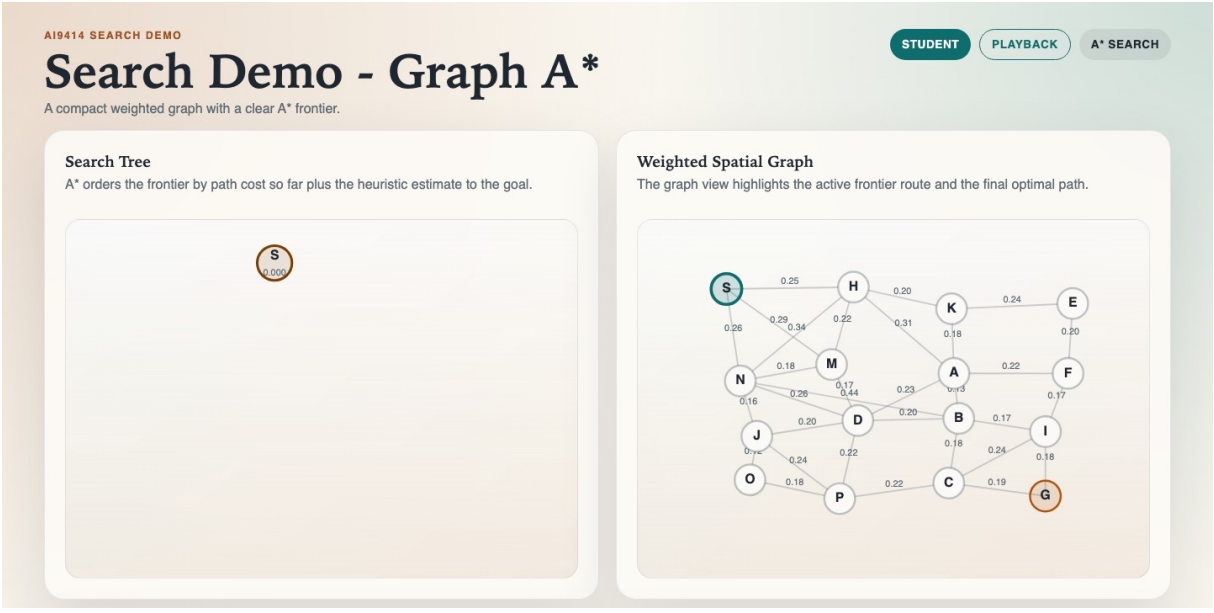

Figure 2: The `graph-astar` demo at the initial state. The left panel shows the search tree; the right panel shows the fixed weighted graph.

3 Search Algorithms

3.1 Depth-First Search

Depth-first search (DFS) follows one branch of the graph as far as it can before backtracking. The frontier behaves like a stack: the most recently discovered unexpanded node is explored next.

- Data structure: stack, or recursion.
- Priority rule: last in, first out.
- Guarantees: complete on a finite graph if repeated states are handled; not guaranteed to find a shortest path.
- Demo: `ai9414 demo graph-dfs`.

In the `graph-dfs` demo, neighbours are returned in sorted node-id order. If you implement DFS with an explicit Python stack, push neighbours in reverse order if you want the alphabetically first neighbour to be expanded first.

3.2 Breadth-First Search

Breadth-first search (BFS) expands all depth-0 nodes, then all depth-1 nodes, and so on. The frontier behaves like a queue: the earliest discovered unexpanded node is explored next.

- Data structure: queue, for example `collections.deque`.
- Priority rule: first in, first out.
- Guarantees: in an unweighted graph, BFS finds a shallowest path in number of edges.
- Demo: `ai9414 demo graph-bfs`.

3.3 Uniform-Cost Search

Uniform-cost search (UCS) generalises BFS to positive edge costs. Instead of expanding the oldest frontier path, it expands the frontier path with the lowest cost so far, usually written as $g(n)$.

- Data structure: priority queue, for example `heapq`.
- Priority rule: lowest $g(n)$.
- Guarantees: with positive edge costs, UCS finds an optimal path to the goal.
- Demo: `ai9414 demo graph-ucs`.

3.4 A* Search

A* search uses both the path cost so far and a heuristic estimate to the goal. It expands the frontier path with the lowest value of

$$f(n) = g(n) + h(n),$$

where $h(n)$ estimates the remaining cost from node n to the goal.

- Data structure: priority queue.

- Priority rule: lowest $g(n) + h(n)$.
- Guarantees: if the heuristic is admissible, A* finds an optimal path.
- Demo: `ai9414 demo graph-astar`.

The A* stub imports a helper for the straight-line distance heuristic:

```
from ai9414.graph_astar import heuristic_to_goal
```

This heuristic is admissible for the supplied spatial graphs.

Algorithm	Frontier discipline	Graph type	Main guarantee
DFS	stack	unweighted	finds a path if one is reached
BFS	queue	unweighted	shallowest path by edge count
UCS	priority by g	weighted	lowest-cost path
A*	priority by $g + h$	weighted	lowest-cost path with admissible h

Table 1: Comparison of the four demos used in this tutorial.

4 Using the Live Python Workflow

The demos include two modes. In `playback` mode, the supplied reference solver is used to show the algorithm. In `live Python` mode, your local Python code receives the graph and returns a trace for the browser to display.

4.1 Download the starter code

1. Start one demo, for example:

   ```
   ai9414 demo graph-bfs --example small
   ```

2. In the browser, click `Download Python stub`.
3. Unzip the downloaded folder.
4. In a second terminal, activate the same virtual environment.
5. Install the requirements if needed:

   ```
   python -m pip install -r requirements.txt
   ```

6. Open `solve_graph.py` and find the TODO section.

The starter file handles the local web connection. Your job is to implement the search function and return a dictionary in the required shape.

4.2 Run your solver

Keep the demo server running in the first terminal. In the folder containing `solve_graph.py`, start the local solver:

```
python solve_graph.py
```

Then return to the browser, switch `Mode` to `live Python`, and click `Solve with Python`. If your returned dictionary is valid, the browser will replay your trace.

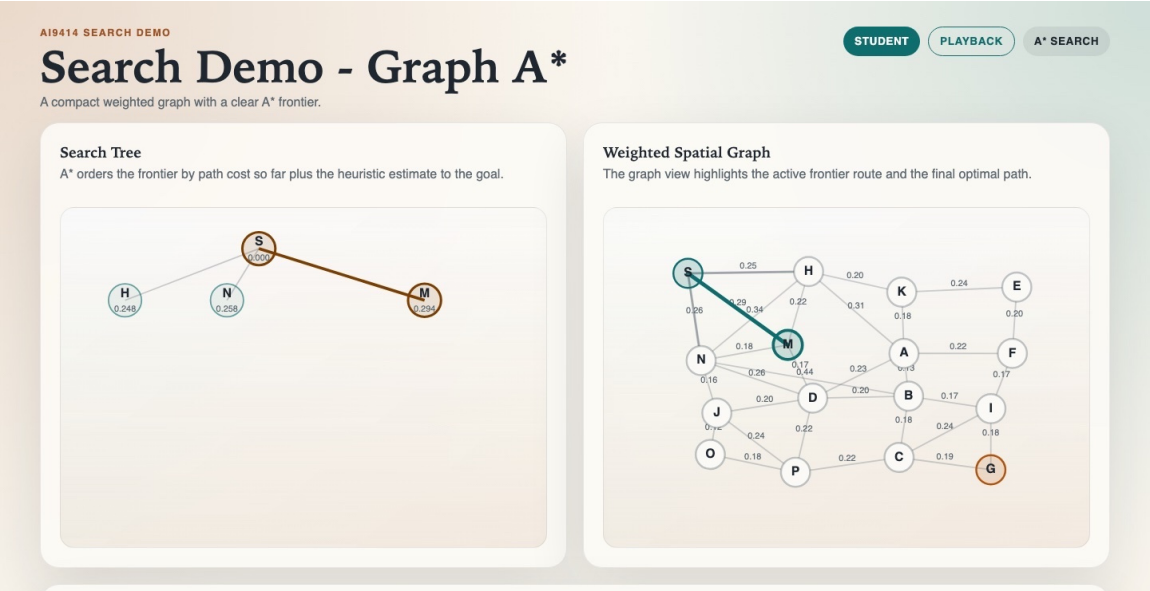

Figure 3: A later A* step. The tree records the frontier history while the graph highlights the active route and explored edges.

4.3 Result dictionaries

The downloaded stub contains the exact validation rules. The key fields are:

```
# DFS result
{
    "algorithm": "dfs",
    "status": "found",        # or "not_found"
    "trace": trace,
    "path": path,
    "visited_order": visited_order,
}

# BFS result
{
    "algorithm": "bfs",
    "status": "found",        # or "not_found"
    "trace": trace,
    "path": path,
    "visited_order": visited_order,
}

# UCS or A* result
{
    "algorithm": "ucs",        # or "astar"
    "status": "found",        # or "not_found"
    "trace": trace,
    "path": path,
    "best_cost": best_cost,
    "visited_order": visited_order,
}
```

The trace events differ slightly by algorithm:

- DFS: `start`, `expand`, `backtrack`, `found`, `fail`.
- BFS: `start`, `expand`, `found`, `fail`.
- UCS/A*: `start`, `expand`, `consider_edge`, `relax`, `found`, `fail`.

Trace schema quick reference. Every trace entry needs a consecutive integer `step`, an `action`, a `node`, a `parent`, and a non-negative integer `depth`. The remaining fields depend on the algorithm:

- DFS entries also need `stack`, the current DFS stack or recursive path.
- BFS entries also need `route`, the path from the start to the current node.
- UCS entries also need `path_cost`, `current_path`, `current_cost`, `best_path`, `best_cost`, and `considered_edge`.
- A* entries also need `path_cost`, `current_path`, `current_cost`, `heuristic`, `priority`, `best_cost`, and `considered_edge`.

Use `None` in Python for optional fields such as a missing parent, missing best cost, or no currently considered edge. The downloaded stub checks these fields before the browser replays your trace.

Counting convention. For DFS and BFS, report the number of nodes reached or discovered, because these algorithms mark nodes as visited when they are first reached. For UCS and A*, report the number of expanded nodes, meaning the number of times a node is removed from the priority queue for expansion. If you use a different convention, state it next to your table.

5 Implementation Hints

5.1 Inspecting neighbours

Use the helper supplied by the relevant demo rather than manually parsing the edge list every time. For DFS and BFS:

```
from ai9414.graph_dfs import get_neighbours
# or:
from ai9414.graph_bfs import get_neighbours

for neighbour in get_neighbours(graph, node_id):
    ...
```

For UCS and A*, neighbours include edge costs:

```
from ai9414.graph_ucs import get_neighbours
# or:
from ai9414.graph_astar import get_neighbours

for neighbour, edge_cost in get_neighbours(graph, node_id):
    ...
```

5.2 Priority queues

Python's `heapq` module implements a min-priority queue. Store tuples so that the first item is the priority:

```
import heapq

frontier = []
heapq.heappush(frontier, (0.0, start_node))
cost, node = heapq.heappop(frontier)
```

For UCS, the priority should be the new path cost:

```
new_cost = current_cost + edge_cost
heapq.heappush(frontier, (new_cost, neighbour))
```

For A*, add the heuristic estimate:

```
from ai9414.graph_astar import heuristic_to_goal

new_cost = current_cost + edge_cost
priority = new_cost + heuristic_to_goal(graph, neighbour)
heapq.heappush(frontier, (priority, new_cost, neighbour))
```

Keep a `best_costs` dictionary for UCS and A*. A later route to the same node should be ignored unless it improves the best known cost to that node.

6 Your mission

1. **Environment check.** Create a virtual environment, install `ai9414`, run `ai9414 list`, and launch all four graph demos. Record the start and goal nodes shown in the `small` example for each demo.

2. **DFS solver.** Download the `graph-dfs` Python stub and implement the TODO in `solve_dfs`. Submit the returned `path`, `visited_order`, and a short explanation of where DFS backtracks.

3. **BFS solver.** Download the `graph-bfs` Python stub and implement `solve_bfs`. Compare the BFS path length with the DFS path length on the same `small` graph. Explain why BFS has the shallowest path guarantee here.

4. **UCS solver.** Download the `graph-ucs` Python stub and implement `solve_ucs` using `heapq`. Report the final `best_cost` and list the first five expanded nodes.

5. **A* solver.** Download the `graph-astar` Python stub and implement `solve_astar`. Use `heuristic_to_goal` for the heuristic term. Compare the number of expanded nodes with UCS on the same graph.

6. **Large-example comparison.** Repeat BFS, UCS, and A* on the `large` example. Fill in a table with path, path cost where applicable, number of expanded nodes, and whether the returned path is guaranteed to be optimal. Separately, include a short four-algorithm summary table for the `small` examples so DFS, BFS, UCS, and A* can be compared side by side.

7. **Heuristic experiment.** In your A* implementation, temporarily replace the heuristic with zero. Run the solver again. What algorithm does it now behave like? Restore the original heuristic before submission.

8. **Reflection.** In two or three sentences, explain why visualising the search tree and the original graph side by side helps distinguish graph nodes from search-tree nodes.

7 Task checklist

If this was a test or exam, you would be asked to submit:

- your completed `solve_graph.py` files for DFS, BFS, UCS, and A*;
- a short results table comparing the four algorithms on the `small` examples;
- one screenshot of a successful live-Python replay;
- answers to the reflection questions above.

References

[1] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.

[2] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths". *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[3] Python Software Foundation. *`heapq` – Heap queue algorithm*. `https://docs.python.org/3/library/heapq.html`.

[4] Oliver Obst. *`ai9414`: Interactive teaching demos for artificial intelligence*. `https://pypi.org/project/ai9414/`.